ModelTrace — Guia Completo de Desenvolvimento
> Plataforma de Observabilidade, Auditoria e Governança de Decisões de Machine Learning

---

1. Visão e Posicionamento

O que é o ModelTrace

ModelTrace é uma plataforma web que transforma cada decisão gerada por um modelo de machine learning em um evento rastreável, auditável e analisável — do momento da predição até o resultado real meses depois.

O sistema não treina modelos. Ele observa, registra e julga o comportamento deles ao longo do tempo.

Por que existe

Modelos de ML são usados para decidir coisas sérias: crédito negado, aluno em risco, transação bloqueada, paciente priorizado. Mas a maioria das empresas não consegue responder perguntas básicas sobre essas decisões uma semana depois:

- Qual versão do modelo decidiu isso?
- Com quais dados?
- O modelo ainda está se comportando como esperado?
- A decisão se mostrou correta depois?
- Existe viés por grupo?
- O que mudou entre a versão antiga e a nova?

O ModelTrace responde essas perguntas.

Posicionamento de mercado

Ferramenta	O que faz	O que falta
MLflow	Rastreia experimentos e treinamento	Não monitora produção
Weights & Biases	Logs de treino e métricas offline	Não audita decisões reais
Evidently AI	Drift e qualidade de dados	Sem auditoria de decisão individual
Arize AI	Observabilidade em produção	Pago, fechado, complexo
ModelTrace	**Auditoria completa da decisão, do payload ao resultado real**	MVP — sem AutoML, sem deploy

---

2. Stack Técnica

Back-end

FastAPI (Python 3.12+)

Justificativas:
- Alinhado com o ecossistema de Data Science
- Validação nativa com Pydantic v2
- Swagger/OpenAPI gerado automaticamente
- Performance assíncrona com async/await
- Tipagem forte que documenta o código

Libs principais:
```
fastapi
pydantic[email]
motor (driver async para MongoDB)
redis[hiredis]
celery[redis]
passlib[bcrypt]
python-jose[cryptography]
httpx (testes)
pytest + pytest-asyncio
```

Front-end

Next.js 14 (App Router) + TypeScript

Libs principais:
```
tailwindcss
shadcn/ui
@tanstack/react-query
recharts
react-hook-form + zod
date-fns
lucide-react
next-themes (dark mode)
```

Banco de dados

MongoDB Atlas (NoSQL principal)

Justificativas:
- Documentos flexíveis para payloads variáveis por modelo

- Agregações poderosas para métricas
- Índices compostos para filtros rápidos
- Time-series nativo para métricas diárias
- TTL index para política de retenção automática

Cache

Redis via Upstash ou Redis Cloud

Usos:
- Cache de dashboards (TTL: 5 minutos)
- Rate limiting por API key
- Idempotência de requests (request_id)
- Filas de jobs (Celery broker)
- Lock distribuído para jobs únicos

Fila

Celery + Redis

Jobs assíncronos:
- Recalcular métricas após nova predição
- Calcular drift periodicamente
- Gerar alertas automáticos
- Exportar relatórios pesados
- Enviar webhooks

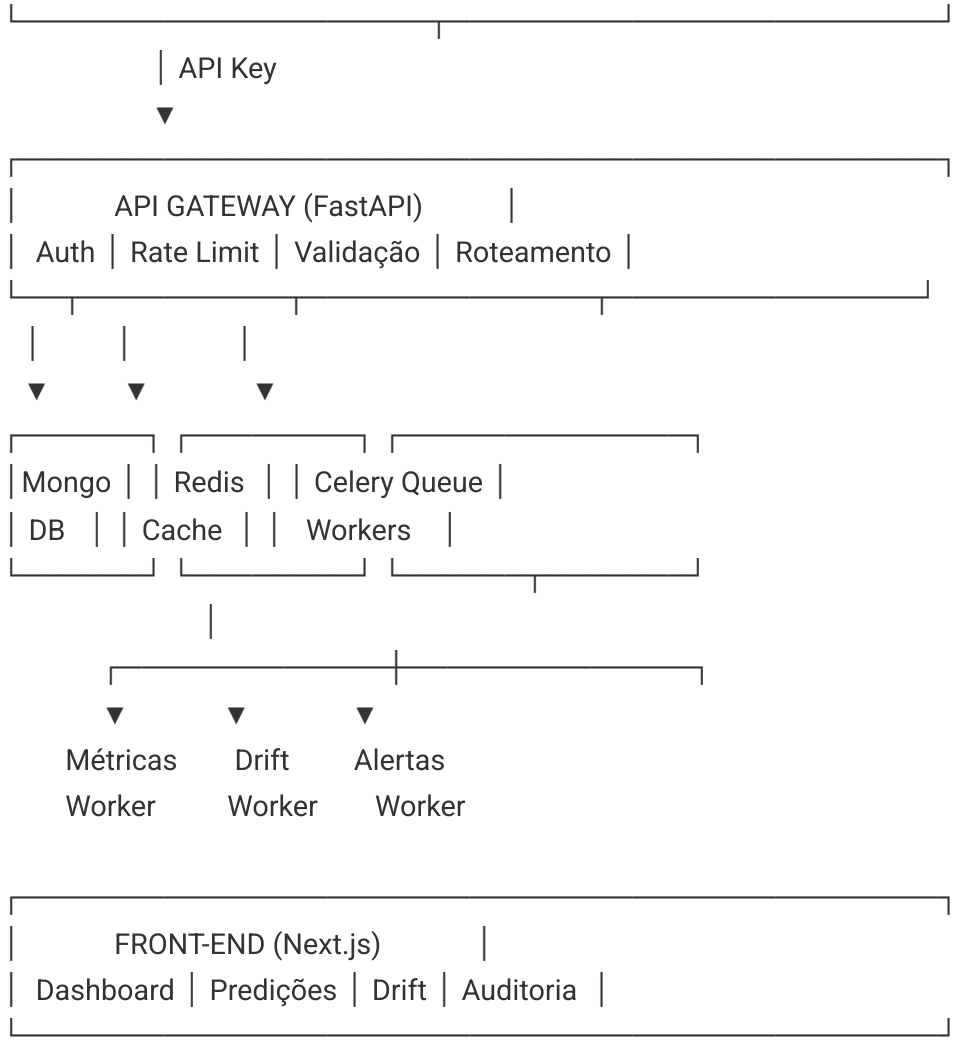
Deploy

```
Docker + Docker Compose (desenvolvimento)
GitHub Actions (CI/CD)
Railway ou Render (deploy)
MongoDB Atlas (banco)
Upstash (Redis)
```

---

3. Arquitetura do Sistema

```
┌─────────────────────────────────────────────────────────────┐
│                    CLIENTES EXTERNOS                        │
│   Scripts Python  │  Pipelines  │  Sistemas                 │
```



Fluxo de ingestão de predição

```
POST /predictions
      |
      ▼
Autenticar API key
      |
      ▼
Validar payload (Pydantic)
      |
      ▼
Verificar idempotência (request_id no Redis)
      |
      ▼
Salvar predição no MongoDB (append-only)
      |
      ▼
Criar evento de auditoria
      |
```

▼
Publicar job na fila Celery
│
▼
Retornar { prediction_id, status: "recorded" }
│
▼ (assíncrono)
Worker atualiza metrics_snapshot do dia
Worker verifica regras de alerta
```

---

## 4. Modelagem NoSQL — Schema Completo

### Coleções

```
users
organizations
projects
models
model_versions
api_keys
predictions
prediction_annotations
prediction_feedbacks
metrics_snapshots
drift_reports
alert_rules
alerts
webhooks
webhook_deliveries
audit_events
exports
```

4.1 organizations

Suporte a multi-tenant básico para deixar o projeto escalável.

```json
{

```
  "_id": "org_abc123",
  "name": "Universidade Federal XYZ",
  "slug": "ufxyz",
  "plan": "pro",
  "owner_id": "user_001",
  "settings": {
    "max_projects": 20,
    "max_predictions_per_day": 100000,
    "retention_days": 730,
    "enable_bias_analysis": true,
    "enable_webhooks": true
  },
  "created_at": "2026-01-15T10:00:00Z",
  "status": "active"
}
```

4.2 users

```json
{
  "_id": "user_001",
  "org_id": "org_abc123",
  "name": "Diego Xavier",
  "email": "diego@ufxyz.edu.br",
  "password_hash": "$2b$12$...",
  "role": "DATA_SCIENTIST",
  "project_permissions": {
    "proj_001": ["READ", "WRITE", "EXPORT"],
    "proj_002": ["READ"]
  },
  "preferences": {
    "theme": "dark",
    "default_project": "proj_001",
    "email_alerts": true
  },
  "created_at": "2026-01-15T10:00:00Z",
  "last_login_at": "2026-04-30T22:10:00Z",
  "status": "active"
}
```

4.3 projects

```json
{
  "_id": "proj_001",
  "org_id": "org_abc123",
  "name": "Academic Dropout Risk",
  "slug": "academic-dropout-risk",
  "description": "Monitor dropout risk predictions for undergraduate students.",
  "problem_type": "binary_classification",
  "domain": "education",
  "owner_id": "user_001",
  "collaborators": ["user_002", "user_003"],
  "tags": ["education", "retention", "classification"],
  "settings": {
    "default_threshold": 0.70,
    "retention_days": 730,
    "enable_drift_monitoring": true,
    "drift_check_frequency": "weekly",
    "alert_email": "mlops@ufxyz.edu.br",
    "bias_sensitive_features": ["scholarship", "night_shift", "age_group"]
  },
  "stats": {
    "total_predictions": 128430,
    "total_models": 2,
    "active_alerts": 3,
    "last_prediction_at": "2026-04-30T23:58:00Z"
  },
  "created_at": "2026-01-15T10:00:00Z",
  "status": "active"
}
```

4.4 models

```json
{
  "_id": "model_001",
  "project_id": "proj_001",
  "org_id": "org_abc123",
  "name": "Dropout Risk Classifier",
  "slug": "dropout-risk-classifier",
```

```
  "description": "Classifies students as high or low dropout risk using academic performance features.",
  "problem_type": "binary_classification",
  "target_name": "dropout",
  "labels": {
    "positive": "high_risk",
    "negative": "low_risk"
  },
  "default_threshold": 0.70,
  "primary_metric": "recall",
  "expected_features": [
    { "name": "semester", "type": "integer" },
    { "name": "attendance_rate", "type": "float", "range": [0, 1] },
    { "name": "failed_subjects", "type": "integer" },
    { "name": "avg_grade", "type": "float", "range": [0, 100] },
    { "name": "credits_completed", "type": "integer" },
    { "name": "scholarship", "type": "boolean" },
    { "name": "night_shift", "type": "boolean" }
  ],
  "production_version_id": "version_002",
  "created_by": "user_001",
  "created_at": "2026-01-20T10:00:00Z",
  "status": "active"
}
```

4.5 model_versions

```json
{
  "_id": "version_002",
  "model_id": "model_001",
  "project_id": "proj_001",
  "version": "v1.2.0",
  "algorithm": "XGBoost",
  "status": "production",
  "created_by": "user_001",
  "created_at": "2026-03-10T09:00:00Z",
  "promoted_to_production_at": "2026-03-15T09:00:00Z",
  "deprecated_at": null,
  "features": ["semester", "attendance_rate", "failed_subjects", "avg_grade", "credits_completed", "scholarship", "night_shift"],
  "hyperparameters": {
    "n_estimators": 300,
    "max_depth": 6,
```

```
    "learning_rate": 0.05,
    "subsample": 0.8
  },
  "training_metadata": {
    "dataset_name": "academic_records_2021_2025",
    "train_size": 42000,
    "test_size": 9000,
    "train_period": { "start": "2021-01-01", "end": "2024-12-31" },
    "validation_strategy": "temporal_split"
  },
  "offline_metrics": {
    "roc_auc": 0.89,
    "pr_auc": 0.63,
    "precision": 0.73,
    "recall": 0.84,
    "f1": 0.78,
    "brier_score": 0.12
  },
  "changelog": "Added scholarship and night_shift features. Improved recall by 3pp vs v1.1.0.",
  "notes": "Trained after feedback analysis showed false negatives concentrated in scholarship students."
}
```

4.6 predictions (coleção central)


```json
{
  "_id": "pred_9f3a2b",
  "project_id": "proj_001",
  "org_id": "org_abc123",
  "model_id": "model_001",
  "model_version_id": "version_002",
  "model_version": "v1.2.0",
  "entity": {
    "id": "student_9821",
    "type": "student",
    "display_name": null
  },
  "created_at": "2026-04-30T20:30:00Z",
  "features": {
    "semester": 2,
    "attendance_rate": 0.64,
    "failed_subjects": 3,
```

```
    "avg_grade": 58.2,
    "credits_completed": 18,
    "scholarship": false,
    "night_shift": true
  },
  "prediction": {
    "label": "high_risk",
    "score": 0.82,
    "threshold": 0.70,
    "above_threshold": true,
    "score_band": "very_high"
  },
  "explanations": [
    { "feature": "failed_subjects", "value": 3, "impact": 0.31, "direction": "increases_risk", "rank": 1 },
    { "feature": "attendance_rate", "value": 0.64, "impact": 0.24, "direction": "increases_risk", "rank": 2 },
    { "feature": "night_shift", "value": true, "impact": 0.14, "direction": "increases_risk", "rank": 3 },
    { "feature": "avg_grade", "value": 58.2, "impact": 0.10, "direction": "increases_risk", "rank": 4 }
  ],
  "decision": {
    "action": "send_to_retention_team",
    "status": "triggered",
    "reason": "score_above_threshold",
    "triggered_at": "2026-04-30T20:30:02Z"
  },
  "feedback": {
    "available": false,
    "observed_label": null,
    "observed_value": null,
    "classification_result": null,
    "received_at": null,
    "source": null
  },
  "annotations": [],
  "flags": {
    "is_critical": true,
    "reviewed_by": null,
    "reviewed_at": null
  },
  "metadata": {
    "source_system": "batch_scoring_pipeline",
    "environment": "production",
    "request_id": "req_abc_123",
    "latency_ms": 43
  }
}
```

```

> **Nota de design:** `score_band` é pré-calculado na ingestão para facilitar filtros rápidos sem range query no score.
> Bandas: `very_low` (< 0.3), `low` (0.3–0.5), `medium` (0.5–0.7), `high` (0.7–0.85), `very_high` (> 0.85)

---

### 4.7 prediction_annotations

Sistema de notas por predição — qualquer usuário com permissão pode adicionar contexto.

```json
{
  "_id": "ann_001",
  "prediction_id": "pred_9f3a2b",
  "project_id": "proj_001",
  "user_id": "user_002",
  "type": "analyst_note",
  "content": "Contacted student. Confirmed family financial difficulties. Forwarded to social services.",
  "tags": ["contacted", "social_support"],
  "created_at": "2026-05-02T14:00:00Z"
}
```

Tipos de anotação:
- `analyst_note` — comentário livre
- `false_positive_flag` — marcado como falso positivo pelo analista
- `action_taken` — descreve ação tomada
- `escalated` — escalado para outro nível

4.8 metrics_snapshots

```json
{
  "_id": "metric_day_20260430_model001_v120",
  "project_id": "proj_001",
  "model_id": "model_001",
  "model_version": "v1.2.0",
  "period": {
    "granularity": "day",
    "start": "2026-04-30T00:00:00Z",
    "end": "2026-04-30T23:59:59Z",
    "label": "2026-04-30"
```
```

```
  },
  "counts": {
    "total": 2391,
    "positive": 447,
    "negative": 1944,
    "with_feedback": 803,
    "without_feedback": 1588
  },
  "performance": {
    "accuracy": 0.79,
    "precision": 0.74,
    "recall": 0.83,
    "f1": 0.78,
    "roc_auc": null,
    "brier_score": 0.13
  },
  "confusion_matrix": { "tp": 218, "fp": 77, "tn": 417, "fn": 91 },
  "score_distribution": {
    "mean": 0.43,
    "std": 0.22,
    "p10": 0.13,
    "p25": 0.24,
    "p50": 0.39,
    "p75": 0.64,
    "p90": 0.79,
    "histogram": [
      { "bucket": "0.0-0.1", "count": 142 },
      { "bucket": "0.1-0.2", "count": 381 },
      { "bucket": "0.2-0.3", "count": 509 },
      { "bucket": "0.3-0.4", "count": 412 },
      { "bucket": "0.4-0.5", "count": 298 },
      { "bucket": "0.5-0.6", "count": 217 },
      { "bucket": "0.6-0.7", "count": 189 },
      { "bucket": "0.7-0.8", "count": 141 },
      { "bucket": "0.8-0.9", "count": 78 },
      { "bucket": "0.9-1.0", "count": 24 }
    ]
  },
  "computed_at": "2026-05-01T00:05:00Z"
}
```

4.9 drift_reports

```json
{
  "_id": "drift_apr2026_model001",
  "project_id": "proj_001",
  "model_id": "model_001",
  "model_version": "v1.2.0",
  "reference_period": { "start": "2026-02-01", "end": "2026-02-28", "count": 18200 },
  "current_period": { "start": "2026-04-01", "end": "2026-04-30", "count": 19800 },
  "data_drift": {
    "overall_status": "attention",
    "features": [
      {
        "name": "attendance_rate",
        "type": "continuous",
        "psi": 0.28,
        "ks_statistic": 0.14,
        "mean_ref": 0.74,
        "mean_cur": 0.67,
        "status": "drift_detected"
      },
      {
        "name": "failed_subjects",
        "type": "continuous",
        "psi": 0.09,
        "ks_statistic": 0.06,
        "mean_ref": 1.2,
        "mean_cur": 1.3,
        "status": "stable"
      },
      {
        "name": "night_shift",
        "type": "categorical",
        "psi": 0.11,
        "proportion_ref": 0.31,
        "proportion_cur": 0.38,
        "status": "attention"
      }
    ]
  },
  "prediction_drift": {
    "psi": 0.17,
    "mean_score_ref": 0.40,
    "mean_score_cur": 0.45,
    "positive_rate_ref": 0.18,
```

```
    "positive_rate_cur": 0.22,
    "status": "attention"
  },
  "performance_drift": {
    "recall_ref": 0.83,
    "recall_cur": 0.76,
    "f1_ref": 0.78,
    "f1_cur": 0.71,
    "feedback_sufficient": true,
    "status": "degradation_detected"
  },
  "created_at": "2026-05-01T01:00:00Z"
}
```

4.10 alert_rules (configuráveis por projeto)


```json
{
  "_id": "rule_001",
  "project_id": "proj_001",
  "model_id": "model_001",
  "name": "Recall drop alert",
  "type": "performance_threshold",
  "metric": "recall",
  "operator": "less_than",
  "threshold": 0.72,
  "severity": "high",
  "min_feedback_count": 100,
  "enabled": true,
  "created_by": "user_001",
  "created_at": "2026-02-01T00:00:00Z"
}
```


Tipos de regra:
- `performance_threshold` — métricas caem abaixo de limite
- `volume_drop` — predições caem % em relação à média
- `data_drift` — PSI de feature ultrapassa limite
- `prediction_drift` — taxa de positivos muda muito
- `missing_features` — features ausentes no payload
- `deprecated_version_active` — versão deprecated ainda recebe predições
- `feedback_rate_low` — taxa de feedback abaixo do mínimo

4.11 webhooks

```json
{
  "_id": "wh_001",
  "project_id": "proj_001",
  "org_id": "org_abc123",
  "name": "Slack Alerts Channel",
  "url": "https://hooks.slack.com/services/...",
  "events": ["alert.created", "alert.severity_high", "drift.detected"],
  "secret": "$2b$12$...",
  "status": "active",
  "created_at": "2026-02-01T00:00:00Z",
  "last_delivery_at": "2026-04-30T20:30:05Z",
  "last_delivery_status": "success"
}
```

4.12 audit_events (imutável)

```json
{
  "_id": "audit_001",
  "org_id": "org_abc123",
  "project_id": "proj_001",
  "actor": {
    "type": "user",
    "id": "user_001",
    "email": "diego@ufxyz.edu.br"
  },
  "event_type": "model_version.threshold_updated",
  "resource": {
    "type": "model_version",
    "id": "version_002"
  },
  "diff": {
    "before": { "threshold": 0.70 },
    "after": { "threshold": 0.75 }
  },
  "context": {
```

```
    "ip": "177.42.0.10",
    "user_agent": "Mozilla/5.0...",
    "api_key_id": null
  },
  "created_at": "2026-04-30T20:15:00Z"
}
```

Índices MongoDB

```js
// predictions — filtros principais
db.predictions.createIndex({ org_id: 1, project_id: 1, created_at: -1 })
db.predictions.createIndex({ model_id: 1, model_version: 1, created_at: -1 })
db.predictions.createIndex({ "entity.id": 1, project_id: 1, created_at: -1 })
db.predictions.createIndex({ "prediction.score": -1, project_id: 1 })
db.predictions.createIndex({ "prediction.label": 1, model_id: 1 })
db.predictions.createIndex({ "prediction.score_band": 1, project_id: 1 })
db.predictions.createIndex({ "feedback.available": 1, model_id: 1 })
db.predictions.createIndex({ "feedback.classification_result": 1, model_id: 1 })
db.predictions.createIndex({ "flags.is_critical": 1, project_id: 1 })
db.predictions.createIndex({ "metadata.request_id": 1 }, { unique: true, sparse: true })
db.predictions.createIndex({ created_at: 1 }, { expireAfterSeconds: 63072000 }) // TTL 2 anos

// metrics_snapshots
db.metrics_snapshots.createIndex({ model_id: 1, "period.granularity": 1, "period.start": -1 })

// audit_events
db.audit_events.createIndex({ project_id: 1, created_at: -1 })
db.audit_events.createIndex({ "actor.id": 1, created_at: -1 })
db.audit_events.createIndex({ event_type: 1, created_at: -1 })

// alerts
db.alerts.createIndex({ project_id: 1, status: 1, severity: 1, created_at: -1 })
```

---

5. Features do Sistema

5.1 Autenticação e Controle de Acesso

Fluxo:

- Registro com confirmação de email (opcional no MVP)
- Login com JWT (access token 15min + refresh token 7 dias)
- Refresh automático no front-end via interceptor
- API Keys com hash bcrypt (apenas o prefixo é legível: `mt_live_abc...`)

Papéis e permissões:

Ação	Owner	Admin	Data Scientist	Analyst	Viewer	API Client
Criar projeto	✅	✅	❌	❌	❌	❌
Criar modelo/versão	✅	✅	✅	❌	❌	❌
Enviar predição	✅	✅	✅	❌	❌	✅
Ver payload completo	✅	✅	✅	Parcial	❌	❌
Exportar dados	✅	✅	✅	❌	❌	❌
Gerenciar usuários	✅	✅	❌	❌	❌	❌
Ver auditoria	✅	✅	✅	Parcial	❌	❌
Anotar predição	✅	✅	✅	✅	❌	❌
Configurar alertas	✅	✅	✅	❌	❌	❌
Configurar webhooks	✅	✅	❌	❌	❌	❌

5.2 Ingestão de Predições

Endpoint principal:
```
POST /api/v1/predictions
Authorization: Bearer mt_live_xxx
```

Payload:
```json
{
  "project_id": "proj_001",
  "model_id": "model_001",
  "model_version": "v1.2.0",
  "entity_id": "student_9821",
  "entity_type": "student",
  "features": { ... },
  "prediction": {
    "label": "high_risk",
    "score": 0.82,
    "threshold": 0.70
  },
  "explanations": [ ... ],
```

```
  "decision": { ... },
  "metadata": {
    "source_system": "batch_pipeline",
    "environment": "production",
    "request_id": "req_unique_123"
  }
}
```

Ingestão em batch (nova feature):
```
POST /api/v1/predictions/batch
```
Aceita até 500 predições por request. Retorna array de resultados com status individual por item. Útil para pipelines noturnos.

Validações:
- Projeto, modelo e versão existem e estão ativos
- API key tem permissão para o projeto
- Score entre 0 e 1
- Label pertence aos labels configurados no modelo
- Threshold entre 0 e 1
- Features não estão vazias
- request_id único (verificado no Redis com TTL de 24h)
- Versão deprecated recebe alerta se ambiente = production

5.3 Feedback Posterior

```
POST /api/v1/predictions/{id}/feedback
```

```json
{
  "observed_label": "dropout",
  "observed_value": 1,
  "feedback_date": "2026-07-15T00:00:00Z",
  "source": "academic_records",
  "notes": "Student did not enroll in second semester."
}
```

Ao receber feedback, o sistema:

1. Atualiza o campo `feedback` na predição
2. Calcula `classification_result`: `tp`, `fp`, `tn`, `fn`
3. Cria evento de auditoria
4. Publica job para recalcular metrics_snapshot do período

5.4 Anotações por Predição

Qualquer predição pode receber anotações de analistas.

```
POST /api/v1/predictions/{id}/annotations
```

```json
{
  "type": "analyst_note",
  "content": "Student contacted. Enrolled in tutoring program.",
  "tags": ["contacted", "tutoring"]
}
```

Usos:
- Documentar ação tomada após decisão
- Marcar como falso positivo após revisão manual
- Registrar escalação para outro time
- Deixar contexto para auditoria futura

5.5 🆕 Prediction Replay

Feature diferencial. Permite re-executar uma predição salva contra uma versão diferente do modelo para ver qual decisão teria sido tomada.

```
POST /api/v1/predictions/{id}/replay
```

```json
{
  "target_version": "v1.3.0",
  "hypothetical_features": null
}
```

```

O sistema não tem o modelo em si — ele registra que o replay foi solicitado e o usuário pode enviar o novo score resultante. O resultado é exibido lado a lado com a predição original:

| Campo | Original (v1.2.0) | Replay (v1.3.0) |
|---|---|---|
| Score | 0.82 | 0.71 |
| Label | high_risk | high_risk |
| Acima do threshold? | ✅ | ✅ |
| Decisão | triggered | triggered |

Caso o score tivesse sido diferente o suficiente para mudar a decisão, o sistema destaca com alerta visual.

---

### 5.6 🆕 What-If Analysis

Interface interativa que permite ao analista modificar features de uma predição salva e simular como o score mudaria — **sem precisar do modelo real**.

Funciona com uma regressão linear local simples calculada a partir das `explanations` (impactos de cada feature). É uma aproximação, não uma predição real.

Exibe aviso claro:

> ⚠️ Esta é uma estimativa baseada nos impactos SHAP registrados. Não substitui uma inferência real do modelo.

Utilidade real: ajuda analistas a entender sensibilidade do score a cada variável, especialmente útil para contestações de decisão.

---

### 5.7 🆕 Análise de Viés por Grupo

Quando o projeto configura `bias_sensitive_features` (ex: `scholarship`, `night_shift`, `age_group`), o sistema calcula métricas separadas por grupo e exibe comparativo.

Exemplo de output:

| Grupo | Total | Positivos | Precision | Recall | F1 |
|---|---|---|---|---|---|
| Bolsistas | 1.200 | 340 (28%) | 0.61 | 0.88 | 0.72 |
| Não-bolsistas | 8.100 | 1.620 (20%) | 0.78 | 0.81 | 0.79 |
| Noturno | 3.400 | 920 (27%) | 0.65 | 0.85 | 0.74 |
```

| Diurno | 5.900 | 1.040 (18%) | 0.80 | 0.80 | 0.80 |

O sistema calcula:
- **Disparate Impact Ratio**: taxa de positivos do grupo A ÷ grupo B (< 0.8 é sinal de atenção)
- **Equal Opportunity Difference**: diferença de recall entre grupos
- **Predictive Parity Difference**: diferença de precision entre grupos

Exibe aviso metodológico:

> ⚠️ Diferença de métricas entre grupos não implica necessariamente viés causal. Analise junto com o contexto de negócio.

5.8 🆕 Webhooks

Quando eventos relevantes ocorrem, o sistema chama URLs externas configuradas pelo usuário.

Eventos disponíveis:
- `alert.created`
- `alert.severity_changed`
- `drift.detected`
- `performance.degraded`
- `prediction.high_risk_batch` (quando >N predições críticas em X horas)
- `feedback.rate_low`

Payload enviado:
```json
{
  "event": "alert.created",
  "project_id": "proj_001",
  "timestamp": "2026-04-30T20:30:00Z",
  "data": { ... },
  "signature": "sha256=..."
}
```

O sistema armazena histórico de deliveries com status HTTP, latência e corpo de resposta. Retry automático (3x com backoff exponencial) em caso de falha.

5.9 🆕 Exportação de Dados

```
```

POST /api/v1/exports
```

```json
{
  "type": "predictions",
  "filters": {
    "model_id": "model_001",
    "model_version": "v1.2.0",
    "date_from": "2026-04-01",
    "date_to": "2026-04-30",
    "label": "high_risk",
    "feedback_available": true
  },
  "format": "csv",
  "include_features": true,
  "include_explanations": true
}
```

Formatos: `csv`, `json`, `parquet` (opcional)

O job de exportação roda em background. O usuário recebe o link para download quando pronto. O arquivo fica disponível por 48h no storage.

5.10 Dashboard de Monitoramento

Cards principais:
- Total de predições (período selecionado)
- Volume hoje vs. média dos últimos 7 dias (com delta %)
- Taxa de alto risco
- Recall atual (com feedback suficiente)
- Alertas ativos por severidade
- Taxa de feedback disponível

Gráficos:
- Série temporal de volume (por dia/semana/mês)
- Distribuição de scores (histograma)
- Evolução de precision/recall no tempo
- Top 5 features mais impactantes (média de |impact|)
- Matriz de confusão do período
- Taxa de decisão positiva por dia
```

Seção de alertas ativos com severidade visual e link para resolução.

Últimas predições críticas (score > 0.85 ou is_critical = true).

5.11 Comparação de Versões

Seletor de Versão A vs. Versão B com filtro de período.

Componentes:
- Cards comparativos de métricas (com delta colorido: verde melhora, vermelho piora)
- Histogramas de score sobrepostos
- Matrizes de confusão lado a lado
- Tabela de todas as métricas
- Top features por versão

Aviso metodológico automático se os períodos de uso forem diferentes:

> ⚠️ v1.1.0 foi usado entre Jan-Fev e v1.2.0 entre Mar-Abr. A comparação pode ser influenciada por sazonalidade ou mudança no perfil dos alunos. Considere filtrar pelo mesmo período se houver sobreposição.

5.12 Drift Monitoring

Tipos calculados:
- Data Drift por feature (PSI + KS test)
- Prediction Drift (distribuição do score)
- Performance Drift (quando há feedback suficiente)

Frequência: semanal (via Celery beat)

Interface:
- Seletor de período de referência e período atual
- Tabela de features com semáforo (verde/amarelo/vermelho)
- Gráfico de PSI histórico por feature
- Score geral de estabilidade do modelo

5.13 Alertas

Configuração por projeto:
- Regras configuráveis via UI (tipo, métrica, operador, threshold, severidade)

- Janela de cálculo configurável (últimas 24h, 7d, 30d)

Gestão:
- Reconhecer alerta (com comentário obrigatório)
- Resolver alerta (fecha o ciclo)
- Ignorar com justificativa
- Escalar

Notificações:
- Email (via configuração de SMTP)
- Webhook (Slack, Discord, Teams, custom)

---

6. API Completa

Autenticação
```
POST   /auth/register
POST   /auth/login
POST   /auth/refresh
POST   /auth/logout
GET    /auth/me
PATCH  /auth/me
```

Projetos
```
GET    /projects
POST   /projects
GET    /projects/{id}
PATCH  /projects/{id}
DELETE /projects/{id}
GET    /projects/{id}/stats
GET    /projects/{id}/dashboard
GET    /projects/{id}/alerts
GET    /projects/{id}/audit-events
GET    /projects/{id}/exports
```

Modelos
```
GET    /projects/{id}/models
POST   /projects/{id}/models
GET    /models/{id}
```

PATCH /models/{id}
GET   /models/{id}/versions
POST   /models/{id}/versions
GET   /models/{id}/metrics
GET   /models/{id}/versions/compare?a=v1.1.0&b=v1.2.0
GET   /models/{id}/score-distribution
GET   /models/{id}/confusion-matrix
GET   /models/{id}/drift
POST   /models/{id}/drift/run
GET   /models/{id}/bias-analysis
```

### Predições
```
POST   /predictions
POST   /predictions/batch
GET   /predictions (com filtros e cursor pagination)
GET   /predictions/{id}
POST   /predictions/{id}/feedback
POST   /predictions/{id}/annotations
GET   /predictions/{id}/annotations
POST   /predictions/{id}/replay
GET   /predictions/{id}/whatif
GET   /predictions/{id}/audit-trail
```

### Alertas
```
GET   /alert-rules
POST   /alert-rules
PATCH /alert-rules/{id}
DELETE /alert-rules/{id}
GET   /alerts
GET   /alerts/{id}
POST   /alerts/{id}/acknowledge
POST   /alerts/{id}/resolve
POST   /alerts/{id}/ignore
```

### Webhooks
```
GET   /projects/{id}/webhooks
POST   /projects/{id}/webhooks
PATCH /webhooks/{id}
DELETE /webhooks/{id}
```
```

```
GET    /webhooks/{id}/deliveries
POST   /webhooks/{id}/test
```

Exportações
```
POST   /exports
GET    /exports
GET    /exports/{id}
GET    /exports/{id}/download
```

API Keys
```
POST   /projects/{id}/api-keys
GET    /projects/{id}/api-keys
DELETE /api-keys/{id}
```

---

7. Telas do Front-end

7.1 Login / Registro
- Layout minimalista em duas colunas
- Lado esquerdo: branding + frase de posicionamento + visual do produto
- Lado direito: formulário
- Dark mode por padrão

7.2 Dashboard Geral
- Header com seletor de projeto e período
- Grid de cards de KPIs
- Gráfico de volume temporal
- Histograma de scores
- Alertas ativos
- Feed de últimas predições críticas

7.3 Lista de Projetos
- Cards com: nome, domínio, total de predições, alertas ativos, última predição, status do modelo em produção
- Filtro por status e domínio

7.4 Detalhe do Projeto (tabs)
- **Overview** — estatísticas gerais, modelos cadastrados, saúde do projeto
- **Models** — lista de modelos com versão em produção e métrica principal
- **Predictions** — tabela com filtros

- **Alerts** — alertas com gestão inline
- **Audit Logs** — timeline de eventos
- **Settings** — configurações, API keys, webhooks, membros

7.5 Detalhe do Modelo (tabs)
- **Overview** — métricas da versão em produção, volume, health score
- **Versions** — histórico de versões com changelog
- **Predictions** — tabela filtrada pelo modelo
- **Performance** — gráficos de precision/recall/F1 no tempo
- **Drift** — relatório de drift mais recente
- **Bias** — análise de viés por grupo sensível
- **Compare** — comparação entre versões

7.6 Detalhe da Predição (tela mais importante)

Estrutura:
1. **Header** — ID, entidade, data, modelo/versão, badge de label e score
2. **Score visual** — gauge/dial mostrando score com threshold marcado
3. **Decisão tomada** — action, status, razão, timestamp
4. **Feature importance** — gráfico horizontal de barras com impact de cada feature
5. **What-If interativo** — sliders para features numéricas, toggles para booleanas
6. **Feedback real** — seção com resultado observado (ou campo para adicionar)
7. **Anotações** — thread de comentários de analistas
8. **Replay** — botão para iniciar um replay contra outra versão
9. **Payload raw** — JSON colapsável do request original
10. **Audit trail** — timeline de todos os eventos ligados a essa predição

7.7 Tela de Drift
- Seletor de período (referência vs. atual)
- Semáforo de estabilidade geral
- Tabela de features com PSI, médias, status
- Gráfico de distribuição sobreposto por feature (referência vs. atual)
- Prediction drift (score distribution comparison)
- Performance drift (se feedback disponível)
- Alertas gerados por drift

7.8 Tela de Bias Analysis
- Seletor de feature sensível
- Tabela comparativa por grupo
- Métricas de fairness (DIRatio, Equal Opportunity Diff, Predictive Parity Diff)
- Explicação de cada métrica com linguagem acessível
- Aviso metodológico em destaque

7.9 Tela de Alertas
- Filtros: severidade, tipo, status, modelo

- Cards com ícone de severidade, título, modelo, data
- Modal de detalhe com histórico do alerta
- Ações inline: reconhecer / resolver / ignorar

7.10 API Playground (dentro do projeto)
- Interface tipo Postman minimalista
- Preenche automaticamente o endpoint, headers e projeto
- Permite testar envio de predição com payload editável
- Exibe response formatada
- Botão "Use esta predição" para salvar como exemplo

---

8. Identidade Visual e UX

Direção estética: Industrial + Técnico

- Paleta: fundo `#0A0A0F` (quase preto), primário `#00D4FF` (ciano elétrico), acento `#FF6B35` (laranja para alertas), success `#00C88C`
- Tipografia: `JetBrains Mono` para dados/scores, `DM Sans` para corpo
- Scores e thresholds em destaque monospace
- Badges de label com cores semânticas: vermelho para high_risk, verde para low_risk
- Drift e alertas com semáforos visuais consistentes
- Tabelas com density alta (modo compacto) e modo expandido
- Gráficos com tooltips ricos ao hover

---

9. Roteiro de Desenvolvimento

Sprint 0 — Fundação (3 dias)

Objetivo: ambiente pronto, repositório configurado, nada de feature ainda.

Tarefas:
- [ ] Criar monorepo: `modeltrace/backend`, `modeltrace/frontend`, `modeltrace/scripts`
- [ ] `docker-compose.yml` com MongoDB + Redis + Backend + Frontend
- [ ] Setup FastAPI com estrutura de pastas, lifespan, CORS, health check
- [ ] Setup Next.js 14 com App Router, Tailwind, shadcn/ui, dark mode
- [ ] Conexão MongoDB com Motor (async)
- [ ] Conexão Redis
- [ ] `.env.example` documentado
- [ ] GitHub Actions: lint + type check em PR
- [ ] README inicial

Entregável: `docker-compose up` sobe tudo, `/health` retorna 200, home page renderiza.

Sprint 1 — Autenticação (4 dias)

Objetivo: usuário pode criar conta, fazer login e acessar área autenticada.

Back-end:
- [ ] Schema `users` no MongoDB
- [ ] `POST /auth/register` com validação de email e hash bcrypt
- [ ] `POST /auth/login` retorna access_token (15min) + refresh_token (7d)
- [ ] `POST /auth/refresh` rotaciona tokens
- [ ] `GET /auth/me` retorna usuário autenticado
- [ ] Middleware de autenticação JWT
- [ ] Middleware de autorização por role
- [ ] Rate limiting básico no Redis (10 tentativas de login por minuto)

Front-end:
- [ ] Tela de login e registro
- [ ] Interceptor de Axios/fetch para refresh automático
- [ ] Proteção de rotas com middleware Next.js
- [ ] Contexto de usuário global
- [ ] Header com nome do usuário e logout

Testes:
- [ ] Login com credenciais válidas → 200 + tokens
- [ ] Login com senha errada → 401
- [ ] Request sem token → 401
- [ ] Refresh com token expirado → 401

Sprint 2 — Projetos e Modelos (4 dias)

Objetivo: usuário pode criar projetos, modelos e versões.

Back-end:
- [ ] CRUD de projetos com validação de ownership
- [ ] CRUD de modelos vinculados a projeto
- [ ] CRUD de versões com validação de status
- [ ] Regra: apenas uma versão `production` por modelo (ou aviso se houver duas)
- [ ] Evento de auditoria em toda criação/alteração
- [ ] Schema de validação Pydantic para cada entidade

Front-end:
- [ ] Lista de projetos (cards)
- [ ] Formulário de criação de projeto
- [ ] Detalhe do projeto (tabs: Overview, Models)
- [ ] Lista de modelos
- [ ] Formulário de criação de modelo
- [ ] Lista de versões com changelog
- [ ] Formulário de criação de versão com campos de métricas offline

Testes:
- [ ] Criar projeto → aparece na lista
- [ ] Usuário sem permissão → 403
- [ ] Versão em produção aparece destacada

Sprint 3 — Ingestão de Predições (5 dias)

Objetivo: sistema externo pode enviar predição e ela aparece no front-end.

Back-end:
- [ ] Schema de API keys (geração, hash, prefixo legível)
- [ ] CRUD de API keys
- [ ] `POST /predictions` com validação Pydantic completa
- [ ] Verificação de idempotência via Redis (request_id)
- [ ] Cálculo automático de `score_band`
- [ ] Evento de auditoria na ingestão
- [ ] `POST /predictions/batch` (até 500 items)
- [ ] `GET /predictions` com cursor pagination e filtros
- [ ] `GET /predictions/{id}`

Front-end:
- [ ] Tela de gerenciamento de API keys (criar, revogar)
- [ ] Lista de predições com tabela, filtros e paginação
- [ ] Badges de label e score_band com cores semânticas
- [ ] Tela de detalhe da predição (blocos 1-5: header, score gauge, features, explanations, decisão)

Scripts:
- [ ] `scripts/seed_predictions.py` — gera 50k predições simuladas com fórmula de risco
- [ ] Populate database com dados realistas para demonstração

Testes:
- [ ] Payload válido → 201 + prediction_id
- [ ] Score fora de range → 422 com mensagem clara
- [ ] request_id duplicado → 409

- [ ] API key inválida → 401
- [ ] Versão inexistente → 404

Sprint 4 — Feedback e Métricas (5 dias)

Objetivo: predições recebem feedback real e o sistema calcula métricas.

Back-end:
- [ ] `POST /predictions/{id}/feedback`
- [ ] Cálculo automático de `classification_result` (tp/fp/tn/fn)
- [ ] Job Celery: recalcula `metrics_snapshot` ao receber feedback
- [ ] `GET /models/{id}/metrics` — métricas agregadas por período
- [ ] `GET /models/{id}/confusion-matrix`
- [ ] `GET /models/{id}/score-distribution`
- [ ] `GET /projects/{id}/dashboard` — dados do dashboard principal

Cache:
- [ ] Cache Redis para `/dashboard` e `/metrics` (TTL 5min)
- [ ] Invalidação de cache quando feedback chega

Front-end:
- [ ] Seção de feedback na tela de detalhe da predição
- [ ] Formulário de adição de feedback
- [ ] Dashboard principal completo (cards + gráficos)
- [ ] Gráficos com Recharts: série temporal, histograma, matriz de confusão
- [ ] Métricas por versão na tela do modelo

Scripts:
- [ ] `scripts/seed_feedback.py` — adiciona feedback em ~40% das predições
- [ ] Simula distribuição realista de tp/fp/tn/fn

Testes:
- [ ] Feedback → classification_result correto
- [ ] Dashboard carrega de cache na segunda requisição
- [ ] Métricas consistentes com contagem manual

Sprint 5 — Drift e Alertas (5 dias)

Objetivo: sistema detecta mudanças de comportamento e gera alertas.

Back-end:

- [ ] Serviço de cálculo de PSI por feature
- [ ] Serviço de cálculo de KS test (scipy)
- [ ] Job Celery: gera `drift_report` semanalmente
- [ ] CRUD de `alert_rules`
- [ ] Job Celery: verifica regras de alerta após cada snapshot
- [ ] Gera `alerts` quando regra violada
- [ ] `POST /alerts/{id}/acknowledge`, `/resolve`, `/ignore`
- [ ] `GET /models/{id}/drift`

Front-end:
- [ ] Tela de drift (semáforo, tabela de features, gráficos)
- [ ] Tela de alertas com gestão inline
- [ ] Badge de alertas ativos no header
- [ ] Seção de alertas no dashboard

Scripts:
- [ ] `scripts/simulate_drift.py` — altera distribuição de `attendance_rate` nos últimos 30 dias
- [ ] Gera queda artificial de recall para testar alerta de performance

Testes:
- [ ] PSI > 0.25 → drift_detected
- [ ] Recall < threshold configurado → alerta criado
- [ ] Alerta acknowledge → status muda, evento de auditoria criado

---

Sprint 6 — Features Avançadas (6 dias)

Objetivo: implementar features diferenciais.

Anotações:
- [ ] `POST /predictions/{id}/annotations`
- [ ] `GET /predictions/{id}/annotations`
- [ ] Thread de anotações na tela de detalhe

Comparação de versões:
- [ ] `GET /models/{id}/versions/compare?a=v1.1.0&b=v1.2.0`
- [ ] Tela de comparação com delta colorido e aviso metodológico

What-If Analysis:
- [ ] Endpoint que retorna estimativa baseada nos impacts
- [ ] Interface com sliders e toggles na tela de detalhe

Bias Analysis:
- [ ] Job que calcula métricas por grupo de feature sensível

- [ ] `GET /models/{id}/bias-analysis`
- [ ] Tela de análise de viés

Webhooks:
- [ ] CRUD de webhooks
- [ ] Envio com assinatura HMAC-SHA256
- [ ] Retry com backoff
- [ ] Histórico de deliveries
- [ ] `POST /webhooks/{id}/test`

Exportação:
- [ ] Job de exportação em CSV
- [ ] Download via signed URL

Sprint 7 — Auditoria e Segurança (3 dias)

Objetivo: auditoria completa e segurança robusta.

Back-end:
- [ ] `GET /projects/{id}/audit-events`
- [ ] `GET /predictions/{id}/audit-trail`
- [ ] Garantir que auditoria cobre: criação de projeto/modelo/versão, alteração de threshold, ingestão de predição, feedback, alertas, API keys
- [ ] Middleware de proteção contra payload gigante (max 1MB)
- [ ] Middleware de proteção contra NoSQL injection (validação de tipos)
- [ ] Auditoria de login e logout

Front-end:
- [ ] Tela de audit logs com filtros por tipo e ator
- [ ] Audit trail na tela de detalhe da predição (timeline visual)
- [ ] Tela de settings com gestão de membros e permissões

Sprint 8 — Qualidade e Deploy (5 dias)

Objetivo: sistema pronto para produção e portfólio.

Testes:
- [ ] 100% dos endpoints críticos com testes de integração
- [ ] Testes de carga: 10k predições em batch e medir latência
- [ ] Verificar uso de índices com `explain()` no MongoDB
- [ ] Testes de segurança: autenticação, autorização, rate limiting

Deploy:
- [ ] Dockerfile multi-stage para backend (tamanho mínimo)
- [ ] Dockerfile para frontend
- [ ] `docker-compose.prod.yml` com variáveis de ambiente separadas
- [ ] Deploy Railway/Render com variáveis de ambiente
- [ ] MongoDB Atlas com usuário de produção (sem IP whitelist aberta)
- [ ] Upstash Redis

Documentação:
- [ ] Swagger completo e documentado
- [ ] `README.md` completo com arquitetura, como rodar, exemplos
- [ ] `docs/architecture.md` com decisões técnicas justificadas
- [ ] `docs/data-model.md` com schema e índices
- [ ] `docs/api.md` com exemplos de payload
- [ ] Seed script documentado
- [ ] `.env.example` completo

Portfólio:
- [ ] Limpeza visual do front-end
- [ ] Screenshots de todas as telas principais
- [ ] Vídeo de demonstração (3-5 minutos) mostrando fluxo completo
- [ ] README com badges: Python, FastAPI, MongoDB, Redis, Next.js, Docker

---

10. Estrutura de Pastas

```
modeltrace/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   └── routes/
│   │   │       ├── auth.py
│   │   │       ├── projects.py
│   │   │       ├── models.py
│   │   │       ├── versions.py
│   │   │       ├── predictions.py
│   │   │       ├── feedback.py
│   │   │       ├── annotations.py
│   │   │       ├── alerts.py
│   │   │       ├── drift.py
│   │   │       ├── webhooks.py
```

```
│   │   │   ├── exports.py
│   │   │   └── audit.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── dependencies.py
│   │   │   └── exceptions.py
│   │   ├── db/
│   │   │   ├── mongo.py
│   │   │   └── redis.py
│   │   ├── models/          # Pydantic schemas (request/response)
│   │   ├── services/        # Lógica de negócio
│   │   │   ├── auth.py
│   │   │   ├── predictions.py
│   │   │   ├── metrics.py
│   │   │   ├── drift.py
│   │   │   ├── alerts.py
│   │   │   ├── bias.py
│   │   │   └── webhooks.py
│   │   ├── workers/         # Celery tasks
│   │   │   ├── celery_app.py
│   │   │   ├── metrics_tasks.py
│   │   │   ├── drift_tasks.py
│   │   │   ├── alert_tasks.py
│   │   │   └── webhook_tasks.py
│   │   └── tests/
│   │       ├── conftest.py
│   │       ├── test_auth.py
│   │       ├── test_predictions.py
│   │       ├── test_metrics.py
│   │       └── test_drift.py
│   ├── Dockerfile
│   ├── pyproject.toml
│   └── .env.example
│
├── frontend/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   └── register/
│   │   └── (dashboard)/
│   │       ├── projects/
│   │       ├── models/
│   │       ├── predictions/
│   │       ├── alerts/
```

```
│   │       └── settings/
│   ├── components/
│   │   ├── ui/              # shadcn/ui components
│   │   ├── charts/          # Recharts wrappers
│   │   ├── predictions/
│   │   ├── models/
│   │   └── shared/
│   ├── features/            # feature slices
│   ├── hooks/
│   ├── lib/
│   │   ├── api.ts           # API client
│   │   └── utils.ts
│   ├── types/
│   ├── Dockerfile
│   └── package.json
│
├── scripts/
│   ├── seed_predictions.py
│   ├── seed_feedback.py
│   ├── simulate_drift.py
│   ├── load_test.py
│   └── generate_api_key.py
│
├── docs/
│   ├── architecture.md
│   ├── data-model.md
│   ├── api.md
│   └── decisions.md
│
├── docker-compose.yml
├── docker-compose.prod.yml
└── README.md
```

---

11. Script de Seed — Fórmula de Risco Simulado

```python
import random
import math

def simulate_risk_score(features: dict, version: str) -> float:
    """
    Simula score de risco de evasão baseado em features acadêmicas.
```

```
    Não é um modelo real — é uma fórmula para gerar dados de demonstração.
    """
    f = features

    # Normalização
    att_norm = 1 - f["attendance_rate"]          # maior ausência = maior risco
    grade_norm = 1 - (f["avg_grade"] / 100)      # menor nota = maior risco
    fail_norm = min(f["failed_subjects"] / 5, 1)  # até 5 reprovações
    credit_norm = 1 - min(f["credits_completed"] / 120, 1)

    # Pesos base
    score = (
        0.35 * fail_norm
        + 0.28 * att_norm
        + 0.20 * grade_norm
        + 0.10 * credit_norm
        + 0.04 * (0.3 if f.get("night_shift") else 0)
        + 0.03 * (0.2 if not f.get("scholarship") else 0)
    )

    # Versão mais antiga tem calibração pior
    if version == "v1.0.0":
        score = score * 0.95 + random.gauss(0, 0.08)
    elif version == "v1.1.0":
        score = score * 0.97 + random.gauss(0, 0.06)
    else:  # v1.2.0 — mais calibrada
        score = score + random.gauss(0, 0.04)

    return round(max(0.01, min(0.99, score)), 4)

def compute_label(score: float, threshold: float = 0.70) -> str:
    return "high_risk" if score >= threshold else "low_risk"

def simulate_explanations(features: dict, score: float) -> list:
    """Gera impactos SHAP simulados coerentes com a fórmula."""
    impacts = {
        "failed_subjects": 0.35 * min(features["failed_subjects"] / 5, 1),
        "attendance_rate": 0.28 * (1 - features["attendance_rate"]),
        "avg_grade": 0.20 * (1 - features["avg_grade"] / 100),
        "credits_completed": 0.10 * (1 - min(features["credits_completed"] / 120, 1)),
        "night_shift": 0.04 * 0.3 if features.get("night_shift") else 0,
        "scholarship": 0.03 * 0.2 if not features.get("scholarship") else 0,
    }

    total = sum(impacts.values()) or 1
```

```
    explanations = []
    for feat, raw_impact in sorted(impacts.items(), key=lambda x: -x[1]):
        if raw_impact > 0.001:
            explanations.append({
                "feature": feat,
                "value": features[feat],
                "impact": round(raw_impact / total * score, 4),
                "direction": "increases_risk"
            })
    return explanations[:5]

def simulate_real_dropout(score: float) -> bool:
    """
    Simula se o aluno realmente evadiu baseado no score.
    Adiciona ruído para criar falsos positivos e falsos negativos realistas.
    """
    base_prob = score * 0.85
    noise = random.gauss(0, 0.15)
    return (base_prob + noise) > 0.5
```

---

12. Testes de Carga

```python
# scripts/load_test.py
import asyncio
import httpx
import time

async def send_prediction(client, api_key, payload):
    r = await client.post(
        "/api/v1/predictions",
        json=payload,
        headers={"Authorization": f"Bearer {api_key}"}
    )
    return r.status_code, r.elapsed.total_seconds()

async def run_load_test(n_predictions=1000, concurrency=50):
    results = []
    async with httpx.AsyncClient(base_url="http://localhost:8000") as client:
        sem = asyncio.Semaphore(concurrency)

        async def bounded_send(payload):
```

```
            async with sem:
                return await send_prediction(client, API_KEY, payload)

        tasks = [bounded_send(generate_payload(i)) for i in range(n_predictions)]
        start = time.time()
        results = await asyncio.gather(*tasks)
        elapsed = time.time() - start

    success = sum(1 for s, _ in results if s == 201)
    latencies = [t for _, t in results]

    print(f"Total: {n_predictions}")
    print(f"Sucesso: {success} ({success/n_predictions*100:.1f}%)")
    print(f"Tempo total: {elapsed:.2f}s")
    print(f"Throughput: {n_predictions/elapsed:.1f} req/s")
    print(f"Latência P50: {sorted(latencies)[int(len(latencies)*0.5)]:.3f}s")
    print(f"Latência P95: {sorted(latencies)[int(len(latencies)*0.95)]:.3f}s")
    print(f"Latência P99: {sorted(latencies)[int(len(latencies)*0.99)]:.3f}s")
```

---

13. Diferenciais de Portfólio

O que torna este projeto excepcional

1. Domínio não trivial
Não é um todo-list. Não é um e-commerce. É um sistema de governança de ML — área que existe em grandes empresas mas raramente em projetos de portfólio.

2. Justificativa técnica para cada decisão
- NoSQL não foi escolhido porque é moderno — foi escolhido porque o schema de `features` é variável por modelo
- Eventos imutáveis não são enfeite — são a base de auditoria confiável
- metrics_snapshots não são redundantes — são a estratégia para escalar sem degradar o dashboard
- request_id no Redis não é detalhe — é proteção real contra duplicação

3. Consciência metodológica
O sistema avisa quando a comparação de versões pode ser injusta. Avisa quando drift não implica necessariamente piora. Avisa quando explicações SHAP não são causalidade. Isso mostra maturidade técnica e científica.

4. Features que refletem problemas reais
- Bias analysis — regulatório em crédito, saúde e educação
- Webhook — integração com ferramentas existentes (Slack, PagerDuty)
- What-If — ferramenta real de contestação de decisões algorítmicas
- Replay — comparação contrafactual entre versões

5. Arquitetura escalável por design, não por acidente
- Append-only predictions
- Agregações materializadas
- Cursor pagination
- Async processing
- Cache estratégico

Como apresentar no portfólio

Título: ModelTrace — ML Decision Auditing Platform

Tagline: "Every model decision. Traced, audited, monitored."

O que destacar em ordem:
1. Problema de negócio real (governança de ML)
2. Arquitetura event-driven com NoSQL (justificativa técnica)
3. Fluxo completo: predição → feedback → métricas → drift → alerta
4. Features diferenciais: bias analysis, what-if, replay
5. Escalabilidade: 100k predições/dia com snapshots e cache
6. Documentação técnica de riscos metodológicos

Frases para README:
> "A decision without an audit trail is just a guess you can't defend."
> "Built for teams that need to answer 'why did your model do that?' six months later."

---

14. Checklist Final de Qualidade

Funcional
- [ ] Fluxo completo: criar projeto → modelo → versão → enviar predição → feedback → ver métricas
- [ ] Batch de predições funciona
- [ ] Drift detectado com dados simulados
- [ ] Alerta gerado automaticamente
- [ ] Webhook disparado e entregue
- [ ] Exportação CSV funcional
- [ ] Auditoria cobre todas as ações críticas
- [ ] Bias analysis mostra diferença entre grupos

Técnico
- [ ] Todos os endpoints com auth e autorização testados
- [ ] Índices verificados com explain() — sem collscans em queries principais
- [ ] Cache sendo usado (log de hit rate)
- [ ] Rate limiting ativo para API keys

- [ ] Payload de 1MB rejeitado com mensagem clara
- [ ] request_id duplicado retorna 409, não 500
- [ ] Jobs Celery rodando e processando

Portfólio
- [ ] Deploy público acessível
- [ ] Seed de dados carregado (dashboard não está vazio)
- [ ] README com arquitetura, stack, como rodar, exemplos
- [ ] Swagger acessível em `/docs`
- [ ] Screenshots de todas as telas
- [ ] Vídeo de demonstração (3-5 min)
- [ ] `docs/architecture.md` com decisões justificadas